RxInsight: Tools and Concepts Primer

Updated 29 October 2026. Learn these concepts in order, then connect them to the source. The implemented stack is Python/PostgreSQL; no frontend framework is needed.

1. Relational databases and constraints

Tables store records. A primary key identifies a row; a foreign key prevents references to missing parent rows. NOT NULL, CHECK and UNIQUE enforce other rules. PostgreSQL provides these controls and executes the analytical SQL.

> Analogy: a spreadsheet can list an invoice for a nonexistent customer. A foreign-key rule makes the database ask for a valid customer record before accepting that reference.

An existing HCP version is not necessarily a temporally correct version. Foreign keys check existence; temporal joins and tests check whether its dates fit the event.

2. SQL and grain

SELECT chooses data; WHERE filters; GROUP BY creates groups; SUM adds measures; JOIN relates tables. Start by explaining one row before combining tables.

```
SELECT territory_key, SUM(trx_count) AS total_trx
FROM warehouse.fact_prescriptions
GROUP BY territory_key;
```

An inner join removes unmatched rows. A left join preserves them with missing right-hand values. A later filter on those right-hand values can accidentally remove the records intended for quarantine.

3. Warehouse, dimensions and facts

Dimensions describe context; facts contain events and measures. The star schema shares date, product, HCP and territory definitions across prescriptions and calls. Grain specifies exactly what one fact row represents.

> Analogy: joining every purchase to every store visit by the shopper can repeat the same sale. Decide whether the question is about purchases, visits or shoppers per month before choosing the join.

The call report aligns facts at person/day before labelling person-months. It does not sum prescription measures after a naive many-to-many raw-fact join.

4. Keys and SCD Type 2

`hcp_id` is the stable source identity. `hcp_key` identifies a warehouse version. SCD Type 2 keeps old attribute versions instead of rewriting them. A June transfer closes the old version the day before the new one begins.

Conflicting same-day or backdated tracked changes are rejected; explicit history backfills are future work. The normal loader remains full-refresh. Study task: explain why longitudinal exposure

follows the person, while historical territory performance follows the version.

5. ETL, staging and rejects

Extract brings records in, transform applies rules, and load writes the target. TEXT staging lets malformed source values reach validation before typed fact loading. Reject files preserve bad records and reasons.

Raw TEXT still needs safe later casts. Guarded PostgreSQL 16 date validation prevents impossible dates from aborting calendar-boundary computation. Type and key checks then quarantine malformed facts.

The two quality questions are whether counts reconcile and whether the correct identities were rejected. The verified full run balances 2,230,247 prescription inputs into 2,229,747 facts and 500 genuine rejects.

6. pandas and NumPy

pandas provides DataFrames; NumPy supports numerical arrays and operations. Transforms parse, mask, merge and group tabular data. Small dimensions stay in memory while facts are processed in batches.

`loc` selects labels and `iloc` selects positions. They coincide on some default indices and diverge after rows are removed. RxInsight tracks positions for rejects, so it uses positional selection.

Missing, finite and integer are different properties. `3.7` is numeric but not a count; infinity is not a usable count either. Explicit checks must happen before conversion or COPY.

7. SQLAlchemy, psycopg2 and COPY

SQLAlchemy supplies supported pandas database-read connections. psycopg2 is the PostgreSQL driver and provides COPY. The engine explicitly selects `postgresql+psycopg2`; a changed generic driver default previously broke fresh installation.

COPY streams buffered records with low command overhead. The eight fact FKs are restored with validation inside the fact transaction. Validation failure prevents an orphan fact load from committing.

8. Transactions

A transaction groups changes that commit or roll back together. Here, fact copying, constraint restoration and reconciliation are protected before the fact commit. Earlier stages have their own commits.

> Analogy: several purchases can belong to one checkout without the whole day's shopping being one transaction. Likewise, protecting fact loading does not make the complete refresh atomic.

Versioned build-and-swap publication would be a future improvement. Do not confuse a normal full-refresh reset with an incremental, restartable production ingestion system.

9. Window functions

Window functions calculate over related rows without always collapsing them. LAG supplies preceding share, RANK handles ties, NTILE creates groups, and SUM OVER produces YTD totals.

The partition expresses business meaning. Brand share needs month and therapeutic area. YTD needs territory and calendar year. Leaving out a partition key can produce a valid SQL result that

answers the wrong question.

10. Indexes and EXPLAIN ANALYZE

A B-tree is an access path, not a precomputed report. Its value depends on filters, coverage, ordering and selected fraction. It also costs writes and storage.

EXPLAIN describes a plan. EXPLAIN ANALYZE executes and measures it. The selective lookup improved from 131.640 ms to 2.250 ms using a Bitmap Heap Scan. The full aggregate stayed a Parallel Seq Scan without material improvement.

> Analogy: a book index helps locate one topic. If you must read the whole book, looking up every index entry may add work. An index is useful because of the access pattern, not simply the book's length.

Inspect estimates versus actual rows, scans, joins and filtering. Discard a warm-up and measure several comparable runs. A warm cache is not proof that an index improved a query.

11. Docker Compose, Make and configuration

Docker packages an environment; Compose describes service settings. This project declares PostgreSQL 16 with host port 5544, a volume and a health check. Make supplies command shortcuts; it does not implement the transformation logic.

Use an isolated database: the normal refresh resets staging and warehouse schemas. `make report` stops on SQL errors. `make test` requires a populated DB and fails instead of silently skipping.

The audit used real PostgreSQL 16.15 through secure transport because local Compose was unavailable. GitHub CI separately ran PostgreSQL containers. This does not verify every local Docker configuration.

12. pytest and CI

The suite has 36 pure unit cases, 21 loaded-data cases and 10 SQL acceptance cases. Both dependency configurations passed 67 with zero skips. CI generates data and runs the actual ETL, rather than presenting offline unit results as a warehouse pass.

A unit checks a small rule; a DB check examines persisted integrity; a report acceptance test checks the business answer. An independent raw-CSV calculation adds evidence using a different algorithm.

13. Study path

Trace one row, write one aggregate, explain one transfer, hand-calculate a denominator and quota, compare two plans, then describe rollback boundaries. Next study incremental ingestion, late-data handling, publication atomicity and workload-driven partitioning as future designs.

Do not claim that sharding, real feeds or causal analysis is implemented. Precise boundaries are more credible than naming extra tools.

Sources: Reviewed source | Verification, architecture and interview questions